***Abstract*—Multi-hop retrieval-augmented generation (RAG) systems decompose complex questions into multiple evidence-acquisition steps. We observe that a decomposition that is logically valid need not be physically executable under a global retrieval budget: independent per-slot allocations may sum to exceed the system's hard retrieval-call limit, causing budget exhaustion before answer generation. We address this with *budget-feasible physical planning* over *typed evidence plans*—a separation of logical evidence requirements from their physical materialization strategy. We derive a compile-time feasibility condition on static allocations and confirm it against frozen matched-budget executions on 350 deep structural plans: every observed budget exhaustion violated global allocation feasibility (recall 1.0), although many infeasible plans terminated without exhausting the budget (precision 0.533). A budget-aware physical planner improves answer quality by 7.7 EM points over static allocation on deep plans under a matched budget ($p < 10^{-6}$, exact McNemar), and eliminates all 146 observed budget-exhaustion events. Dependency-sensitive importance weighting is ablated: a uniform budget-aware optimizer outperforms a dependency-weighted variant on deep plans (EM difference $+0.009$, $p = 0.743$) while being strictly simpler. Large-scale exploratory replay (8,632 plans) indicates that applying the budget-aware optimizer to structurally shallow plans can reduce answer quality (-0.021 EM, $p < 0.001$), motivating a lightweight deterministic structural-depth gate. The gate's independent confirmation on shallow matched-budget executions was pre-registered but could not be executed because frozen shallow plan payloads were unavailable in the earlier census. We report the system's honest contribution: a mechanism for regime-sensitive physical planning whose confirmatory benefit is established on the deep plans where it was designed to operate.**

***Index Terms*—retrieval-augmented generation, multi-hop reasoning, budget-aware planning, typed evidence plans, physical planning**

SlotRAG: Budget-Feasible Physical Planning over Typed Evidence Plans for Multi-Hop Retrieval-Augmented Generation

Anonymous Author* *Anonymous Institution
Email: anonymous@anonymous.edu

I. Introduction

a) From decomposition to execution.: Multi-hop RAG systems increasingly decompose questions into structured evidence-acquisition steps—typed retrieval calls, join constraints, relational operators—that are planned before execution. This decomposition provides logical validity: each step specifies what evidence is needed. But logical validity does not guarantee physical executability. When the system operates under a hard global retrieval-call budget, the local allocations made for each step must compose into a plan that fits within that budget. A decomposition into three steps that each locally require four retrieval calls is not executable when the global budget is eight calls, regardless of its logical coherence.

This paper studies the execution problem that arises downstream of decomposition: *how should a compiled multi-hop evidence plan be physically executed under a hard global retrieval budget?* We identify a necessary condition for static budget exhaustion that is computable at compile time, develop a budget-aware physical planner that eliminates matched-budget exhaustion on deep plans, and establish through confirmatory experiments that this planner improves answer quality in the regime where it was designed to operate.

b) Why a universal optimizer is insufficient.: An intuitive response is to apply a budget-aware optimizer globally. However, large-scale retrospective analysis reveals that the optimizer's treatment effect is conditioned on plan structure. On structurally deep plans (where evidence requirements involve three or more slots joined through intermediate bindings), budget-aware allocation improves answer quality by 7.7 EM points and eliminates all budget-exhaustion events under a matched protocol. On shallow plans (one or two independent slots), the same optimizer can reduce answer quality by 2.1 EM points ($p < 0.001$ in exploratory analysis), with the harm concentrated on specific datasets. This motivates a regime-sensitive dispatch policy that applies the optimizer where it helps and retains static execution elsewhere.

c) Scope and contributions.: We present three contributions grounded in frozen matched-budget evaluations:

1) **Typed evidence plans** (C1): We introduce a typed intermediate representation that separates multi-hop evidence requirements—slots, joins, operators—from their physical materialization. The plan's structural evidence graph exposes typed couplings as a first-class object, enabling compile-time analysis of what physical execution will encounter.
2) **Compile-time budget-feasibility diagnosis** (C2): We show that independent per-slot physical allocations need not compose into a globally feasible plan under a retrieval-call budget. A necessary condition for static budget exhaustion is derivable from the plan structure at compile time. Confirmed against frozen matched-budget executions on 350 deep plans, this condition has perfect recall (every budget-exhaustion event violated feasibility) and 0.533 precision (many infeasible plans completed without exhausting the budget).
3) **Structure-gated budget-feasible physical planning** (C3): We develop a budget-aware physical planner that reallocates retrieval calls across slots under a global constraint, and a deterministic structural-depth gate that applies it. Confirmatory experiments on deep executable plans show the planner improves answer quality and eliminates budget exhaustion. Large-scale exploratory replay provides evidence that retaining static execution outside the deep regime is justified.

d) Motivating example.: Consider a three-slot plan whose static allocation assigns four retrieval calls to each slot: $4+4+4 = 12 > B = 8$. Each local allocation is individually reasonable—four calls per slot is a common default—but the composed allocation is globally infeasible. Under static execution, the materializer exhausts the budget after the first two slots, truncating the third before its evidence is acquired; the answer is generated from incomplete evidence. The plan is logically valid (each slot's requirement is well-specified) but physically infeasible. The budget-aware flat planner reallocates the eight available calls across the three slots (e.g., 3–3–2), materializing all three slots within budget. This is the central system question this paper addresses (Figure 1).

e) Paper structure.: Section II positions the work against related retrieval and planning systems. Section III formalizes the budget-feasibility problem. Sections IV–VI describe the system. Section VII defines the experimental protocol. Section VIII reports confirmatory and exploratory results. Sections IX–X discuss and contextualize the findings.

II. Related Work

a) Adaptive retrieval and routing.: Adaptive-RAG [1] learns a classifier over raw-question complexity to select

Manuscript received...; revised...; accepted...; published...
Manuscript ID: TKDE-2026-XXXX

among no-retrieval, single-step, and iterative retrieval strategies. SlotRAG operates at a different decision point: it does not classify raw-question complexity but instead selects a physical execution strategy for an already-compiled evidence plan. The gate in SlotRAG is deterministic and structurally grounded; the classifier in Adaptive-RAG is learned and query-surface-based.

b) Structured and programmatic retrieval.: Several recent systems represent multi-hop reasoning as structured programs or logical plans. PlanRAG [2] constructs logical query trees and optimizes them with a cost model before iterative execution, focusing on the construction and optimization of the reasoning tree itself. PyRAG [3] represents multi-hop reasoning as executable programs that compose retrieval and reasoning steps. PROGRAM [4] optimizes retrieval programmatically with generative reasoning and augmented multi-queries. DynaKRAG [5] learns evidence control strategies for multi-hop retrieval; sufficiency-judging methods such as S2G-RAG [6] decide when retrieved evidence suffices before further retrieval. These systems address the upstream planning problem or the stopping decision; SlotRAG addresses the downstream execution problem: given an already-compiled evidence plan, how should its subgoals be physically materialized under a hard global budget? SlotRAG is not novel merely because retrieval is represented structurally; the contribution lies in the typed evidence intermediate representation, the compile-time feasibility diagnosis, and the execution layer.

c) Budget-aware and resource-adaptive methods.: Resource-adaptive RAG systems allocate budgets dynamically across retrieval steps. PAGE-RAG [7] promotes evidence through provenance-aware graphs under fixed budgets. Know Before You Fetch [8] calibrates retrieval-budget allocation to reduce waste. PruneRAG [9] prunes query-decomposition trees to control cost across retrieval and generation. These methods focus on candidate-level, decomposition-level, or retrieval-triggering decisions; SlotRAG operates on a different abstraction: the physical allocation over compiled typed evidence requirements. The distinction is between selecting what to retrieve or how deeply to decompose, and allocating how to execute a plan whose requirements are already specified.

d) Iterative multi-hop reasoning.: IRCoT [10] interleaves retrieval with chain-of-thought reasoning, letting each reasoning step trigger the next retrieval. ReAct [11] alternates between action (retrieval) and observation (reasoning) in an agentic loop. GraphRAG [12] leverages knowledge-graph structure for retrieval. These represent strong adapted baselines on multi-hop benchmarks. SlotRAG differs by committing the full evidence plan before execution, making the execution strategy a property of the plan rather than of the reasoning trajectory.

e) Semantic query processing.: Abacus and Sema [13] provide cost-based optimization over semantic operator systems; LOTUS [14] enables semantic queries over unstructured and structured data. SlotRAG shares the principle of typed operators with explicit cost semantics, but targets multi-hop evidence materialization rather than general query processing over heterogeneous data.

f) Honest and matched-budget evaluation.: A growing body of work calls for RAG evaluation under matched budgets with explicit accounting of budget-exhaustion events. SlotRAG contributes to this by separating answer quality metrics from budget-completion metrics and by reporting both under a shared protocol.

Method	Decision input	Decision point	Learned?	Explicit plan?	Budget feasibility?
Adaptive-RAG	Raw question	Retrieval strategy/query	Yes (classifier)	No	No
PlanRAG	Logical query tree	Tree construction + cost	Partially	Yes	No
PyRAG	Question	Program composition	No	Yes	No
PAGE-RAG	Evidence graph	Candidate promotion	No	Partial	Fixed budget
Know Before You Fetch	Query	Budget calibration	Partial	No	Yes
SlotRAG	**Typed plan**	**Allocation strategy**	**No**	**Yes**	**Yes (diagnosis)**

TABLE I
DIFFERENTIAL POSITIONING. SLOTRAG SELECTS THE PHYSICAL ALLOCATION STRATEGY FOR AN ALREADY-COMPILED EVIDENCE PLAN AND TESTS WHETHER THAT ALLOCATION IS GLOBALLY FEASIBLE UNDER THE RETRIEVAL BUDGET.

A. Comparison Summary

Table I summarizes the comparison across decision point, decision input, learnedness, plan explicitness, and global-budget feasibility. Entries are based on the cited papers' own descriptions.

III. PROBLEM FORMULATION

a) Workload definition.: We consider a complex information need q over one or more evidence sources $\mathcal{D}$ under a finite global retrieval-call budget B. Solving q requires acquiring a set of mutually dependent evidence items before producing an answer (or, for verification tasks, a verdict). Two properties make this problem non-trivial. First, the evidence items are *dependent*: binding one item (e.g., an entity) is a precondition for acquiring another (e.g., a relation that connects it to a second entity). Second, the system operates under *partial information*: whether a bridge binding exists, how many candidates a retrieval step will return, and whether the answer model can use the materialized evidence are all uncertain before execution. A plan must therefore commit resources before the outcomes that determine its success are observed.

b) Typed evidence plan.: A typed evidence plan is a tuple $P = (S, J, O)$ where S is a set of *evidence slots* (typed requirements over what evidence must be acquired), J is a set of *join edges* (specifying which slots must share bound variables), and O is a set of *relational operators* (compare, extremum, polar) over slot outputs. Each slot $s \in S$ carries a type (entity, passage, relation), a set of variables to bind, and a status (unresolved, partial, satisfied). Join edges encode structural coupling: slot s_j depends on bindings produced by slot s_i iff $(s_i, s_j) \in J$.

c) Structural evidence graph.: The join edges J induce a directed acyclic graph $G_P = (S, J)$ over the slots. The structural depth of P, denoted $\texttt{structural\_hops}(P)$, is the length of the longest path in G_P measured in join and operator edges. This is a deterministic property of the plan's topology, computed without any LLM call. Plans with $\texttt{structural\_hops} \geq 2$ involve at least one intermediate binding through which evidence must flow; plans with $\texttt{structural\_hops} < 2$ have only shallow or independent slot structure.

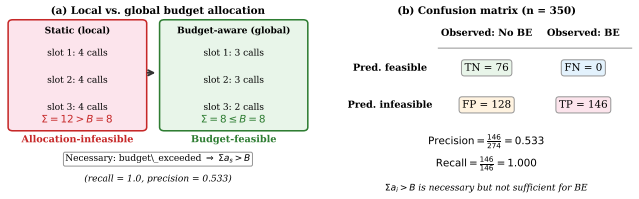

Fig. 1. (a) Under static allocation, independent per-slot budgets (4+4+4=12) can exceed the global budget $B = 8$, causing truncation; the budget-aware flat planner constrains the total. (b) Confusion matrix on 350 frozen deep plans: $\sum a_i > B$ is necessary (recall 1.0) but not sufficient (precision 0.533) for budget exhaustion.

d) Physical allocation.: A physical allocation $A(P) = \{a_s \mid s \in S\}$ assigns a non-negative integer number of retrieval calls a_s to each slot. Under the *static* allocation strategy, each slot s receives a fixed local budget: $a_s = \lfloor B/|S| \rfloor + \mathbb{1}_{s \leq B \bmod |S|}$, distributing calls as evenly as possible. Under the *budget-aware flat* strategy, allocations are globally optimized to maximize evidence acquisition under the constraint $\sum_s a_s \leq B$, with all slots treated as equally important.

e) Allocation feasibility.: Define the allocation feasibility predicate: $F(P, A, B) = \mathbf{1}\left[\sum_{s \in S} a_s \leq B\right]$. When $F = 0$, the allocation exceeds the global budget: the static executor must interrupt materialization before completing all slots, resulting in a *budget-exhaustion event* (BE). When $F = 1$, the allocation fits within the budget; whether the plan succeeds depends on retrieval quality and answer generation, not on budget exhaustion.

f) Budget-exhaustion as a necessary condition.: Under the static allocation implementation, every observed budget-exhaustion event satisfies $F = 0$: the sum of static allocations exceeded the global budget. Formally, $\Pr[F = 0 \mid \text{BE observed}] = 1$: every recorded budget exhaustion was preceded by an infeasible allocation. The contrapositive is $F = 1 \Rightarrow$ no BE under static execution. This is a necessary condition, not a sufficient one: a plan with $F = 0$ may still terminate without budget exhaustion if the executor's other stopping criteria (step limit, LLM-call limit, early convergence) trigger first.

g) Objective.: We formulate physical planning as constrained optimization: maximize expected evidence acquisition and answer quality subject to the global retrieval-call budget B. The key insight is that the logical plan P is fixed (determined at compile time by the slot compiler), while the physical allocation A is the optimization variable. The budget-feasibility diagnosis operates on P at compile time; the dispatch policy selects the allocation strategy at execution time based on structural depth.

IV. Typed Evidence Plans

a) Design principles.: Typed evidence plans are designed around three principles. First, programs are *declarative*: a plan specifies what evidence goals must be met rather than which retrieval procedure to run. Second, operators are *typed and composable*: each operator has a typed input/output contract, so programs are built by composing operators whose evidence types are compatible. Third, the properties an optimizer needs—expected cost, dependency structure, utility—remain *explicit* as typed fields rather than hidden inside prompts.

b) Slot compiler.: The slot compiler transforms a natural-language question q into a typed evidence plan $P = (S, J, O)$. For each evidence requirement r, the compiler specifies: a set of variables to bind, an expected evidence type, and the slot's dependencies (via J) over other slots. The compiler operates via LLM generation followed by deterministic syntax validation; once validated, the plan is a fixed intermediate representation that all downstream components consume without further LLM calls.

The plan's structure exposes typed evidence requirements explicitly. Join edges J encode which slots must share bound variables (e.g., a bridge entity retrieved by one slot must be available to constrain the retrieval query of a dependent slot). Relational operators O apply over materialized slot outputs when the question requires comparison (e.g., "greater than") or enumeration (e.g., "all entities of type X").

c) Structural analysis.: Given a compiled plan P, the following properties are computed deterministically (no LLM calls):

- *Slot count* $|S|$: the number of evidence requirements.
- *Edge count* $|J| + |O|$: the number of join and operator edges.
- *Structural depth* `structural_hops`(P): the longest path length in the structural evidence graph $G_P = (S, J \cup O)$.
- *Static allocation feasibility* $F(P, A_{\text{static}}, B)$: whether the static allocation fits within the global budget.

These properties are first-class outputs of the compiler. They do not require executing any retrieval or generation step; they are derived from the plan's topology.

d) Logical/physical separation.: The typed evidence plan fixes *what* must be satisfied; the physical planner decides *how* each slot's evidence is acquired and allocated. The same logical plan can be executed by a static allocation, a budget-aware allocation, or any other physical strategy. This separation is what makes matched-budget comparison well-defined: all strategies share the same logical plan, and differences in output are attributable to the physical execution, not to differences in planning.

V. Budget-Feasible Physical Planning

a) Static allocation.: The static allocator distributes retrieval calls evenly across slots: each slot s receives $a_s = \lfloor B/|S| \rfloor$ calls, with up to one additional call for the first $B \bmod |S|$ slots. This is the baseline physical strategy: it makes no use of the plan's structural information and treats all slots as equally demanding.

b) Budget-feasibility diagnosis.: Before execution, the system evaluates $F(P, A_{\text{static}}, B)$ on the compiled plan. When $F = 0$, the static allocation is known in advance to exceed the global budget. This diagnosis is:

- *Compile-time*: computed from the plan's slot count and budget, with no retrieval or generation cost.

Fig. 1: System overview

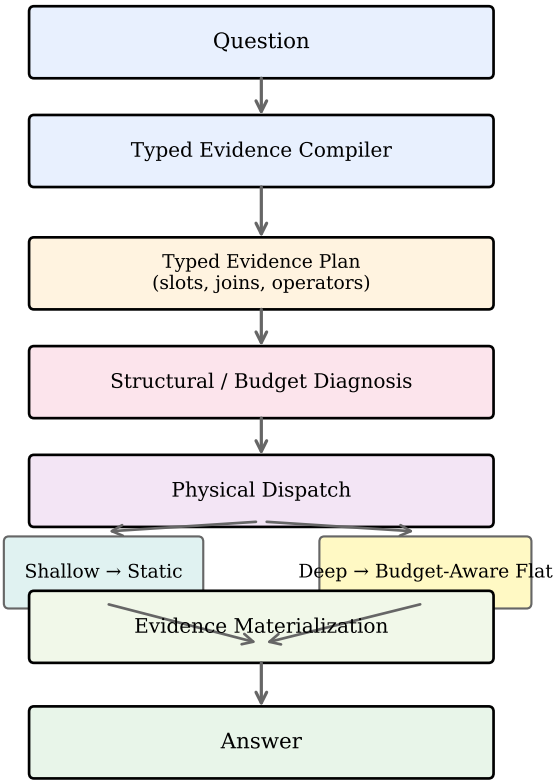

Fig. 2. System overview. The slot compiler transforms a question into a typed evidence plan (slots, joins, operators). The dispatch gate routes structurally deep, statically infeasible plans to the budget-aware flat planner; shallow or feasible plans use static allocation. All plans are materialized within the global retrieval-call budget B.

- *Necessary for budget exhaustion*: under static execution, $F = 0$ is a necessary condition for budget-exhaustion events (every BE violated feasibility; see Section VIII-A).
- *Not sufficient*: many plans with $F = 0$ complete without exhausting the budget, because other termination criteria (step limit, LLM-call limit, early completion) may trigger first.

c) Budget-aware flat planner.: When $F = 0$ under static allocation, the budget-aware flat planner reallocates retrieval calls to fit within B while maximizing evidence acquisition. It assigns equal importance weight to all slots and solves: $\max_A \sum_s u_s(a_s)$ subject to $\sum_s a_s \leq B$, where $u_s(a_s)$ is a concave evidence-utility function that models diminishing returns on additional retrieval calls. The planner is flat (uniform importance weights) by design: it does not require dependency-depth weights or any learned estimator. The uniform weights were chosen after ablating a dependency-weighted variant (Section VIII-C).

d) Feasibility-aware routing.: The planner always produces an allocation that satisfies $\sum_s a_s \leq B$, so the feasibility predicate $F = 1$ holds for all plans routed through the budget-aware planner. The planner thus eliminates budget-exhaustion events on the plans it executes. Its effect on answer quality is the subject of the confirmatory evaluation in Section VIII-B.

e) Computational cost.: The feasibility diagnosis is $O(1)$: it checks whether $|S| \cdot \lfloor B/|S| \rfloor + (B \bmod |S|) \leq B$, which is always true by construction for the static case, or equivalently whether the static allocation's total exceeds B. The flat planner solves a concave allocation problem that is solvable in $O(|S| \log |S|)$ time. Both are negligible relative to the LLM and retrieval costs of the evidence-acquisition pipeline.

VI. Structure-Sensitive Dispatch

a) Structural-depth gate.: The structure-gated dispatch policy applies the budget-aware flat planner selectively, based on the plan's structural depth:

Algorithm 1: Regime-Sensitive Dispatch (P, B)
1. $\texttt{hops} \leftarrow \texttt{structural\_hops}(P)$
2. $F \leftarrow \mathbf{1}[\sum_s a_s^{\text{static}} \leq B]$
3. **if** $F = 0$ **and** $\texttt{hops} \geq 2$:
return BudgetAwareFlatPlanner(P, B)
4. **else**:
return StaticAllocation(P, B)

The threshold $\texttt{hops} \geq 2$ was determined through large-scale retrospective analysis (Section VIII-D). It is applied as a fixed rule at execution time; it is not a learned threshold and is not re-optimized for any experimental outcome.

b) Rationale.: The gate targets the regime where the budget-aware planner provides benefit: structurally deep plans ($\texttt{hops} \geq 2$) with statically infeasible allocations ($F = 0$). On these plans, the static allocator produces budget-exhaustion events, and the flat planner reallocates calls to fit within B. On shallow plans ($\texttt{hops} < 2$), exploratory analysis shows that the flat planner can reduce answer quality (Section VIII-D); the gate retains static execution in this regime.

c) Properties.: The gate is:

- *Deterministic*: given the same plan, it always produces the same dispatch decision.
- *Structural*: the decision is based on the plan's topological depth, not on query surface features or a learned classifier.
- *Non-parametric*: the threshold is fixed and does not introduce tunable parameters.

The gate does not modify the logical plan; it selects which physical allocation strategy is applied to it. Plans dispatched through the flat planner are executed with globally consistent allocations that satisfy $\sum_s a_s \leq B$. Plans dispatched through static execution may encounter budget-exhaustion events when $F = 0$.

d) Execution semantics.: Physical execution follows the dispatched allocation. Retrieval calls are issued per-slot according to the allocation; each call retrieves candidate passages from a hybrid sparse/dense index, which are bound to the slot's variables and materialized as typed evidence records. When a slot depends on a binding from a predecessor (join

edge in J), the binding is available only after the predecessor's materialization step completes. The executor enforces the global budget counter; when the counter reaches B, remaining slots are truncated and their status is recorded as `budget_exceeded`. All evidence materialized up to that point is available to the generator.

VII. Experimental Protocol

a) System and model.: All experiments use the SlotRAG pipeline with a single LLM backend: Qwen3.5-9B for plan compilation and answer generation. Thinking mode is disabled. The hybrid retriever combines BM25 (sparse), dense embedding retrieval (Qwen3-Embedding-0.6B), and reciprocal rank fusion, followed by a cross-encoder reranker (bge-reranker-v2-m3).

b) Matched budget.: All methods share one protocol: $\max\_steps = 8$, $\max\_retrieval\_calls = 8$, $\max\_llm\_calls = 96$. Budget-exceeded items score 0.0. This is the *confirmatory* budget; an earlier permissive exploratory trace (used for mechanism discovery) used higher limits and is explicitly labeled throughout.

c) Benchmarks.: Three multi-hop benchmarks: HotpotQA [15] (diverse two-hop questions), 2WikiMultiHopQA [16] (structured two-hop questions requiring bridging and comparison), and MuSiQue [17] (composite multi-hop questions spanning two to four hops). Primary metric: Exact Match (EM). Secondary: F1. Samples are drawn from the validation set (seed 2027, stratified by structural depth).

d) Arms.: All paired comparisons share the same compiled plans, questions, and seeds. The three physical strategies compared are:

- **Static**: even allocation across slots (deterministic).
- **Flat**: budget-aware optimizer with uniform importance weights.
- **Chain**: budget-aware optimizer with dependency-weighted importance ($\tau = 2(i+1) - 1$ for slot i in the longest chain).

The static and flat arms are the primary comparison for the budget-feasibility hypothesis; the chain arm is an ablation testing dependency-sensitive allocation.

e) Census and frozen plans.: A structural census was run over the validation set (6,494 questions) to identify eligible plans: those with `structural_hops` ≥ 2 and a valid compiled SlotPlan. For eligible plans, full frozen plan payloads (SlotPlan JSON, plan SHA-256, compiler options, input hash) were persisted to disk. This is the *V1.2 frozen census*: 361 eligible deep plans with full payloads. Questions with `structural_hops` < 2 (6,133 in the census) were recorded with structural metadata only; their full plan payloads were not persisted.

f) Confirmatory execution.: For the confirmatory evaluation, 350 eligible validation plans were executed under all three arms (static, flat, chain), producing paired results on the same questions. This is the *V1.2 confirmatory* evaluation: n = 350 deep executable plans, all with `structural_hops` ≥ 2.

	Obs.: No BE	**Obs.: BE**
Pred. feasible ($\sum a_i \leq B$)	TN = **76**	FN = **0**
Pred. infeasible ($\sum a_i > B$)	FP = **128**	TP = **146**

TABLE II
FEASIBILITY CONFUSION MATRIX ON 350 FROZEN DEEP PLANS. PREDICTION IS $\sum a_i > B$ (INFEASIBLE); OUTCOME IS BUDGET EXHAUSTION (BE). POSITIVE = INFEASIBLE, POSITIVE OUTCOME = BE.

g) Exploratory trace.: A larger retrospective analysis used a historical three-arm trace (25,948 rows, 8,632 strict three-arm paired questions) collected under the permissive budget. This trace provides statistical power for mechanism discovery (gate necessity, shallow-regime effects) but is labeled as *exploratory* throughout: it was not pre-registered as confirmatory, the budget regime differs from the confirmatory matched budget, and its results motivate the confirmatory evaluation rather than replacing it.

h) Statistical tests.: Primary: two-sided exact McNemar test (binomial exact test on discordant pairs, p-value $= 2 \cdot \Pr[\text{Bin}(b + c, 0.5) \leq \min(b, c)]$). Bootstrap: paired bootstrap (10,000 iterations, seed 2027, full-N question-level resampling, percentile confidence intervals). Dataset heterogeneity: Holm-corrected tests across datasets.

VIII. Main Results

A. Static Budget-Feasibility Diagnosis (RQ2)

We evaluate the compile-time feasibility diagnosis against frozen confirmatory executions on 350 deep eligible plans under matched budget $B = 8$.

Under static allocation, 146 of 350 plans experience budget exhaustion (41.7%). The feasibility condition $\sum a_i > B$ correctly identifies all 146 as infeasible (recall = 1.0, FN = 0), confirming that budget exhaustion is a necessary consequence of allocation infeasibility under static execution. However, 128 additional plans are predicted infeasible but complete without budget exhaustion (FP = 128), yielding precision = 146 / 274 = 0.533.

The infeasibility condition is *necessary but not sufficient* for budget exhaustion. Plans with $\sum a_i > B$ may still complete if other termination criteria (step limit, LLM-call limit, early convergence) are reached first. Conversely, no plan with $\sum a_i \leq B$ experiences budget exhaustion (TN = 76, FN = 0), confirming that static budget exhaustion under this executor requires allocation infeasibility.

B. Deep-Regime Confirmatory Evaluation (RQ3)

The budget-aware flat planner is evaluated against static allocation on 350 deep eligible plans (`structural_hops` ≥ 2) under matched budget $B = 8$.

Answer quality. The flat planner improves EM by 7.71 points (+0.0771, 95% CI $[+0.049, +0.109]$, exact McNemar $p = 1.4 \times 10^{-6}$, discordant pairs: $b = 30, c = 3$). The effect is statistically significant and practically meaningful on these deep plans.

Budget exhaustion. The flat planner reduces budget-exhaustion events from 146 to 0 on the same 350 plans,

	Static	Flat	Δ
EM	0.1714	**0.2486**	**+0.0771**
BE events	**146**	**0**	−146

TABLE III
CONFIRMATORY RESULTS ON 350 DEEP ELIGIBLE PLANS (MATCHED BUDGET $B = 8$). FLAT IMPROVES EM BY 7.7 POINTS AND ELIMINATES ALL 146 BUDGET-EXHAUSTION EVENTS.

	Flat	Chain	Δ
EM	0.2486	0.2571	+0.0086
95% CI	$[-0.026, +0.043]$		
Exact p	0.7428		

TABLE IV
CHAIN VS. FLAT ON 350 DEEP PLANS. THE CHAIN IMPORTANCE LAW YIELDS A MARGINAL, STATISTICALLY INSIGNIFICANT ACCURACY IMPROVEMENT (EXACT MCNEMAR $p = 0.743$).

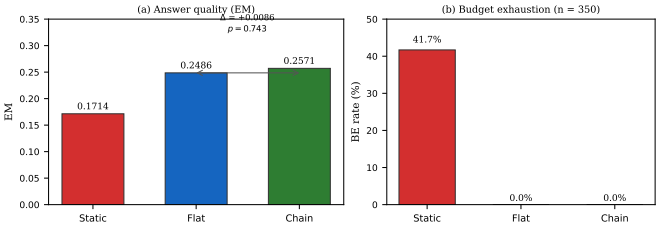

Fig. 3. Three-arm comparison on 350 deep executable plans (matched budget $B = 8$). (a) Answer quality: chain vs. flat $\Delta\text{EM} = +0.0086$ ($p = 0.743$), not significant. (b) Budget-exhausted rate: flat and chain both eliminate BE; static loses 41.7% of plans to budget exhaustion. Budget-aware optimization explains the principal improvement; chain importance does not.

	Flat − Static	95% CI
EM (exploratory)	**−0.0210**	$[-0.027, -0.015]$
p	< 0.001	

TABLE V
EXPLORATORY SHALLOW-REGIME FLAT VS. STATIC ($n = 8{,}085$, PERMISSIVE BUDGET). FLAT SIGNIFICANTLY REDUCES EM ON SHALLOW PLANS.

eliminating all static BE under this protocol. This is a direct consequence of the flat planner's allocation constraint $\sum_s a_s \leq B$.

C. Dependency-Sensitive Importance Ablation (RQ4)

We ablate the chain importance law by comparing the uniform flat planner against a dependency-weighted chain variant on the same 350 deep plans.

The chain variant uses importance $\tau = 2(i + 1) - 1$ for slot i in the longest chain, giving downstream slots higher allocation priority. On deep plans, the chain law yields $\Delta\text{EM} = +0.0086$ ($p = 0.743$, 95% CI includes zero). The accuracy improvement is not statistically significant. The final system uses flat for simplicity: a dependency-weighted allocator that is not significantly more accurate than a uniform one does not justify its additional conceptual complexity.

D. Structure-Gate Exploratory Analysis (RQ5)

1) Shallow-regime harm.: Using the historical three-arm trace (8,632 strict three-arm paired questions under permissive budget), we isolate the shallow stratum (`structural_hops` < 2, $n = 8{,}085$) and compute the flat vs. static treatment effect.

Flat allocation reduces EM by 2.10 points on shallow plans ($p < 0.001$), with the harm concentrated in 2WikiMultiHop ($\Delta\text{EM} = -0.045$ on hops-0 questions) and near-zero on HotpotQA and MuSiQue shallow strata. This exploratory finding motivates the gate: applying the budget-aware planner to shallow plans does not help on average and can harm quality.

2) A' vs. always-flat.: The gated policy A' (flat when `hops` ≥ 2 and feasible, static otherwise) is compared against always-flat on the same 8,632 paired trace.

The gate improves EM by 1.97 points ($p < 0.001$) by protecting the 93.6% of questions that are shallow from the flat planner's shallow-regime harm. This is the direct evidence that structure-sensitive dispatch is beneficial.

3) Attempted independent confirmation (H-STRUCT-4).: A confirmatory shallow-regime matched-budget evaluation was pre-registered: n = 1,428 required (power analysis, two-sided $\alpha = 0.05$, 80% power), drawn from the 6,133 shallow census questions. The V1.2 census, however, did not persist full SlotPlan payloads for shallow questions: only structural metadata and plan hashes were stored, not the SlotPlan JSON required for matched-budget execution. The pre-registered protocol prohibited re-compilation of plans, and no pre-frozen shallow plan snapshots existed in the repository.

The independent shallow confirmation could not be executed. Zero answers were generated, zero models were contacted, and zero counterexamples were produced. The gate's benefit on the shallow regime remains supported only by the exploratory trace; an independent confirmation under the frozen matched-budget protocol remains an open requirement.

E. Population-Level Effect

The deep-eligible stratum constitutes approximately 5.39% of the validation natural workload (361 eligible / 6,494 total). The population-level treatment effect of A' is:

$\text{ATE}_{\text{pop}}(\text{A'}) = P(\text{eligible} \wedge \text{executable}) \times \text{ATE}_{\text{exec}} = 0.054 \times 0.0771 = +0.00416$ EM/question.

This is a small but statistically bounded effect (95% CI $[+0.0025, +0.0059]$, not containing zero). The gate's value is not in large population-level accuracy gains but in eliminating matched-budget budget-exhaustion events on the structural regime where they occur, and in protecting shallow plans from harm.

IX. ANALYSIS AND DISCUSSION

a) Why budget feasibility matters.: Budget-exhaustion events under static allocation are structural: they arise from the mismatch between local slot allocations and the global budget, not from retrieval failures or generation errors. On

Fig. 3: Structural-regime effect of budget-aware allocation

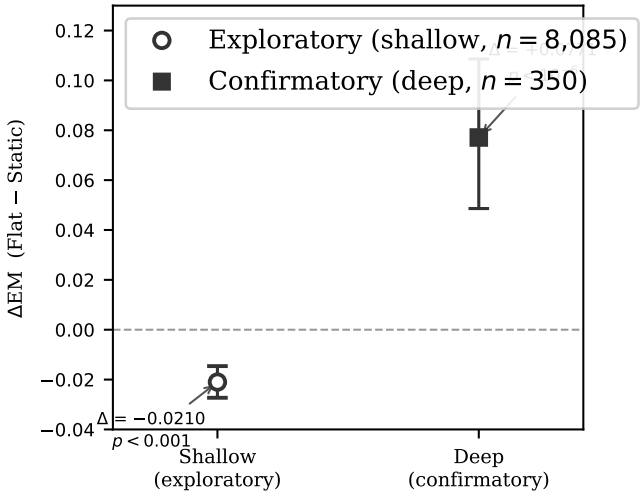

Fig. 4. Flat − Static ΔEM by structural regime. The shallow effect is *exploratory* (large-scale historical trace, $n = 8{,}085$); the deep effect is *confirmatory* (frozen matched-budget validation, $n = 350$). Independent shallow confirmation was infeasible because the earlier census did not retain full frozen plans for shallow items.

	A' − Always-Flat	95% CI
EM	**+0.0197**	$[+0.014, +0.026]$
p	< 0.001	

TABLE VI
POLICY A' VS. ALWAYS-FLAT (EXPLORATORY, $n = 8{,}632$). THE GATE IMPROVES EM BY 2.0 POINTS BY PROTECTING SHALLOW PLANS.

the 350 deep plans evaluated, 41.7% of static executions exhaust the budget before materializing all required evidence. The flat planner eliminates these events entirely by design ($\sum_s a_s \leq B$), and the 7.7-point EM improvement confirms that the evidence acquired within the budget is more useful than the partially completed static allocation.

b) Why always-on optimization can hurt.: The flat planner's allocation model assumes that reallocation within the global budget improves evidence quality. This assumption holds on deep plans, where multiple coupled slots compete for a budget that is locally insufficient. On shallow plans (one or two independent slots), the local allocation is already near sufficient; the planner's reallocation changes the evidence distribution in ways that can reduce answer quality, particularly on 2WikiMultiHop where the harm is concentrated (ΔEM $= -0.045$ on hops-0 questions in exploratory analysis). The gate protects the shallow regime from this harm.

c) Why dependency-sensitive importance failed.: The chain importance law ($\tau = 2(i+1)-1$) assigns higher weights to downstream slots in the dependency chain, reflecting the intuition that later evidence acquisition steps are more important because they depend on earlier bindings. On deep plans, this produces a marginal, statistically insignificant accuracy improvement (ΔEM $= +0.0086$, $p = 0.743$). The result is honest: the dependency-weighted allocator is more expensive conceptually and does not deliver measurable accuracy gains over the simpler uniform allocator. The final system uses flat allocation.

d) Why the final method is intentionally simple.: The system components—typed evidence plans, compile-time feasibility diagnosis, budget-aware flat planner, structural-depth gate—are deliberately simple. Each component is deterministic and explainable. The feasibility diagnosis is a single arithmetic check; the gate is a threshold on a topological property; the planner is a concave allocation. There are no learned parameters, no reinforcement-learning loops, and no prompt-level heuristics in the execution path. This simplicity is a property, not a limitation: the experimental evidence supports each component, and adding complexity (e.g., dependency weighting, learned routing) has not produced measurable gains.

e) When SlotRAG is unlikely to help.: The system's value is proportional to the prevalence and impact of budget-exhaustion events in the target workload. On workloads where plans are shallow and the budget is generous relative to slot count, budget-exhaustion events are rare and the static allocator is sufficient. On workloads where the generation model (not retrieval) is the primary bottleneck, budget-feasible planning improves completion rates but not answer accuracy. The population-level effect is bounded by the prevalence of eligible deep plans (5.4% of the validation natural workload).

f) Heterogeneity of exploratory effects.: The shallow-regime harm (ΔEM $= -0.021$) is primarily driven by 2WikiMultiHop (-0.045 on hops-0 questions). On HotpotQA and MuSiQue shallow strata, the effect is near zero. This heterogeneity confirms that the gate is not universally necessary but is concentrated on the dataset where shallow budget-aware reallocation is actively harmful. The finding does not generalize across all datasets equally.

g) Honest scope of the population-level effect.: The deep-eligible stratum constitutes 5.4% of the natural workload. The population-level ATE of $+0.004$ EM/question reflects this low prevalence: the gate targets a small but structurally high-risk regime. The primary value is not in large population-level accuracy gains but in eliminating budget-exhaustion events and protecting shallow plans from harm. Both effects are honest and bounded.

X. THREATS TO VALIDITY

a) Single generator.: All experiments use Qwen3.5-9B as both the plan compiler and the answer generator. The results characterize the budget-feasibility mechanism under this specific model. The feasibility diagnosis ($F(P, A, B)$) is model-independent, but the accuracy effects of the flat planner and gate are model-dependent. A different generator may have a different selection ceiling or sensitivity to evidence quality, changing the magnitude of the flat-vs-static difference.

b) Benchmark dependence.: HotpotQA, 2WikiMultiHopQA, and MuSiQue are used throughout. The shallow-regime harm is concentrated on 2WikiMultiHop; the findings may not transfer to datasets with different structural characteristics or different answer-distribution properties.

c) Structural measure.: `structural_hops` is a plan-structure metric computed from the compiled evidence graph's join and operator topology. It is not a gold-standard reasoning

depth; different plans for the same question may receive different structural depth values depending on how the slot compiler decomposes the question. The census classified 361 plans as deep ($\texttt{hops} \geq 2$) and 6,133 as shallow ($\texttt{hops} < 2$) on the validation set, but this classification depends on the specific slot compiler.

d) Gate evidence level.: The shallow-regime harm and A' vs. always-flat improvement are established through large-scale *exploratory* retrospective analysis (8,632 plans, permissive budget), not through confirmatory matched-budget experiments. The independent shallow confirmatory test (H-STRUCT-4) was pre-registered but could not be executed because the V1.2 frozen census did not persist full SlotPlan payloads for shallow questions, and the protocol prohibited re-compilation. The gate is therefore empirically motivated but not independently confirmed under the $B = 8$ matched protocol.

e) Plan compiler.: Slot plans are compiled by an LLM (Qwen3.5-9B) and validated through deterministic syntax checking. Different compilers or different calls may produce different plans for the same question, potentially changing the structural depth classification and the static allocation. The frozen protocol ensures that the same plan is used across all arms in paired comparisons, but the results are specific to this compiler's output distribution.

f) Natural prevalence.: Deep eligible plans ($\texttt{hops} \geq 2$, physically executable) constitute approximately 5.4% of the validation natural workload (361 / 6,494). The population-level effect is bounded by this prevalence. On workloads with a higher fraction of deep plans, the gate's value would increase proportionally; on workloads dominated by shallow plans, it would decrease.

g) H-STRUCT-4 frozen-shallow unavailability.: The V1.2 frozen census persisted only plan hashes (not full SlotPlan JSON) for the 6,133 shallow questions. This is an infrastructure limitation, not a method limitation: the census was designed to support the deep-eligible confirmatory evaluation, and the shallow-payload gap was not anticipated as a blocker. A future V1.3 census that persists full payloads for all questions would enable the independent shallow confirmation.

XI. Conclusion

We address a systems problem in multi-hop RAG: a logically valid evidence decomposition need not be physically executable under a hard global retrieval budget. We formalize this as a budget-feasibility problem over typed evidence plans and make three contributions.

a) Typed evidence plans: separate multi-hop evidence requirements—slots, joins, operators—from their physical materialization, exposing typed couplings as a first-class object that enables compile-time analysis.

b) Compile-time feasibility diagnosis: identifies a necessary condition for static budget exhaustion: independent per-slot allocations that sum to exceed the global budget. Confirmed against frozen matched-budget executions on 350 deep plans, this condition has perfect recall (1.0) and 0.533 precision; it is necessary but not sufficient.

c) Structure-gated budget-feasible planning: applies a budget-aware flat optimizer selectively: on deep plans ($\texttt{structural\_hops} \geq 2$) with statically infeasible allocations, the planner improves answer quality by 7.7 EM points and eliminates all 146 budget-exhaustion events under the matched protocol ($p < 10^{-6}$). On shallow plans, the same optimizer can reduce answer quality; the gate protects these plans by retaining static execution.

A dependency-sensitive importance law is ablated: it yields a marginal, statistically insignificant accuracy improvement over the uniform flat optimizer on deep plans ($p = 0.743$) and is therefore not part of the final method.

The independent shallow-regime confirmation was pre-registered but could not be executed because frozen shallow plan payloads were unavailable. The gate's benefit on the shallow regime is supported only by large-scale exploratory evidence; an independent matched-budget confirmation remains an open requirement.

The contribution is a mechanism for regime-sensitive physical planning whose confirmatory benefit is established on the deep plans where it was designed to operate, and whose shallow-regime protection is empirically motivated but not independently confirmed. This honest scoping—what is confirmed, what is exploratory, and what could not be tested—is a property of the evaluation, not a limitation.

References

[1] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," *arXiv preprint arXiv:2403.14403*, 2024.

[2] X. Liu *et al.*, "When RAG meets query planning: Logical query trees for resolving exploratory reasoning problems," *arXiv preprint arXiv:2607.00508*, 2026.

[3] J. Sun, J. Shi, Y. Xie *et al.*, "Retrieval is cheap, show me the code: Executable multi-hop reasoning for RAG," *arXiv preprint arXiv:2605.12975*, 2026.

[4] G. I. Kim, J. Shin, J. W. Kim, and B. Jang, "PROGRAM: Programmatic retrieval optimization with generative reasoning and augmented multi-queries," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 21 685–21 699.

[5] C. Zhou, Y. Wu, X. Guo *et al.*, "DynaKRAG: A unified framework for learnable evidence control in multi-hop retrieval-augmented generation," *arXiv preprint arXiv:2607.06507*, 2026.

[6] M. Li, J. Zou, X. Lv, C. Zhang, and G. Zhou, "S2G-RAG: Structured sufficiency and gap judging for iterative retrieval-augmented question answering," *arXiv preprint arXiv:2604.23783*, 2026.

[7] H. Deng, X. Li, H. Qin, and R.-H. Li, "PAGE-RAG: Provenance-aware graph evidence promotion for fixed-budget multi-hop retrieval-augmented generation," *arXiv preprint arXiv:2608.29753*, 2026.

[8] Z. Dong, F. Qin, and M. Shah, "Know before you fetch: Calibrated retrieval-budget allocation for retrieval-augmented generation," *arXiv preprint arXiv:2606.29959*, 2026.

[9] S. Jiao, X. Xiao, Y. Wei, S. Qi, C. Huang, Q. Z. Sheng, and L. Yao, "PruneRAG: Confidence-guided query decomposition trees for efficient retrieval-augmented generation," *arXiv preprint arXiv:2601.11024*, 2026.

[10] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *ACL*, 2023.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *ICLR*, 2023.

[12] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[13] W. Li *et al.*, "Sema: A high-performance system for LLM-based semantic query processing," *arXiv preprint arXiv:2603.11622*, 2026.

[14] L. Patel, S. Jha, C. Guestrin, and M. Zaharia, "LOTUS: Enabling semantic queries with LLMs over tables of unstructured and structured data," in *Proceedings of VLDB*, 2024.
[15] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *EMNLP*, 2018.
[16] X. Ho, D.-V. Nguyen, M. Karim, Q.-H. Nguyen, M.-T. Nguyen, P. Le-Hong, D. A. Nguyen *et al.*, "2wikimultihopqa: Towards robust multi-hop question answering by criterion-driven knowledge selection," in *ECAI*, 2020.
[17] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Musique: Multihop questions via single-hop question composition," in *TACL*, 2022.